

Procédure de mise à jour

Examens blancs LFJP

Préparer une nouvelle année, publier et déployer

Version de référence : 20 août 2026

Procédure destinée à la personne qui maintient les données et publie l'application.

1. Avant de commencer

Cette application contient des données statiques dans des fichiers TypeScript. Pour préparer une nouvelle session, il faut modifier les données, vérifier les liens et les dates, tester la compilation, puis publier le commit sur GitHub. Vercel reconstruira ensuite automatiquement l'application.

À préparer

- Les dates officielles des épreuves.
- Les listes d'épreuves et leurs classes.
- La liste actualisée des enseignants.
- Les salles, capacités et aménagements.
- Les horaires, jurys et surveillances.
- Les informations à afficher sur l'accueil.

Règle importante

Ne modifiez jamais le dossier `dist` : il est généré automatiquement. Travaillez dans `src`, puis laissez Vite reconstruire `dist`.

2. Mettre à jour l'accueil

- Ouvrir `src/features/home/constants.ts`.
- Modifier les titres, dates, libellés et liens des cartes d'exams dans `HOME_CALLOUT_ENTRIES`.
- Utiliser une date ISO pour le champ `date`, par exemple `2026-12-12`.
- Vérifier que chaque lien correspond à une route réellement déclarée dans `src/app/App.tsx`.

```
dateLabel: "12, 13 et 14 décembre 2026",
date: "2026-12-12",
title: "Baccalauréat blanc 2026",
```

Ajouter une nouvelle carte

Créer une constante `HomeCalloutEntry`, l'ajouter dans `HOME_CALLOUT_ENTRIES`, puis créer la page correspondante si la route n'existe pas encore.

3. Mettre à jour les données

Tableau de bord principal

Modifier `src/features/exam-dashboard/data/dashboard-data.ts`. Ce fichier contient notamment les enseignants, épreuves, salles, missions de surveillance, horaires, indicateurs et aménagements.

Mathématiques et DNB

Modifier ou créer un fichier dans `src/features/math-exam-dashboard/data/datasets`. Le jeu de données doit respecter `MathExamDashboardData` : en-tête, indicateurs, enseignants, surveillance, salles et onglets.

Oraux

Modifier les candidats dans les dossiers `src/features/french-oral-exam-202604`, `french-oral-exam-202605`, `french-oral-exam-202606` ou `grand-oral-exam-202604`. Vérifier les dates ISO, heures, classes, jurys et salles.

Surveillances BAC et DNB

Modifier `src/features/bac-dnb-surveillance/data/scheduleData.ts`. Vérifier les colonnes d'preuves, les horaires et les noms des surveillants. L'heure de présence est calculée automatiquement 30 minutes avant l'preuve.

Contrôles de cohérence

✓ Chaque salle utilisée existe dans la liste des salles.

✓ Chaque enseignant utilisé existe dans l'annuaire.

✓ Les dates sont au format YYYY-MM-DD lorsqu'elles sont utilisées pour le tri.

✓ Les horaires de convocation précèdent les horaires de passage.

✓ Les routes de l'accueil correspondent aux routes React.

4. Tester en local

- 1 Ouvrir un terminal à la racine du projet.
- 2 Installer les dépendances avec `npm install`.
- 3 Lancer `npm run lint` et corriger toute erreur.
- 4 Lancer `npm run build` et vérifier que la compilation réussit.
- 5 Lancer `npm run dev`, ouvrir l'adresse affichée et tester les pages modifiées.

```
npm install
npm run lint
npm run build
npm run dev
```

Test manuel minimal

✓ Ouvrir la page d'accueil.

✓ Cliquer sur chaque carte modifiée.

✓ Recharger directement une URL interne.

✓ Tester les onglets et la recherche.

✓ Tester au moins un export PDF et un export CSV.

✓ Vérifier l'affichage sur ordinateur et mobile.

5. Publier sur GitHub

Une fois les tests terminés, vérifier les fichiers modifiés avant de créer le commit.

```
git status
git add .
git commit -m "Update exam data for 2026"
git push origin main
```

Le push doit se terminer par une confirmation d'envoi vers `origin/main`. Si Git signale un conflit ou un rejet, ne forcez pas le push : récupérez d'abord les changements distants avec `git pull`.

Vérification après publication

```
git status --short --branch
git log --oneline --max-count=2
```

Le résultat attendu est une branche main synchronisée avec origin/main et aucun fichier non suivi ou modifié non pris en compte.

6. Déployer avec Vercel

- 1 Ouvrir le projet Vercel relié au dépôt GitHub.
- 2 Vérifier que la branche de production est main.
- 3 Vérifier que le Framework est Vite, la commande npm run build et le dossier de sortie dist.
- 4 Lancer Redeploy ou attendre le déploiement automatique du nouveau commit.
- 5 Ouvrir le domaine Vercel et tester la page d'accueil ainsi qu'une URL interne.

Configuration des routes

Le fichier vercel.json contient une redirection vers index.html. Il est nécessaire pour que les routes React fonctionnent lors d'un accès direct ou après un rechargement de page.

En cas d'erreur npm

Si Vercel affiche ERESOLVE, vérifier que package.json et package-lock.json sont bien commités ensemble. Le projet utilise ESLint 8 et TypeScript 5.5.4 pour rester compatible avec les outils TypeScript utilisés par l'application.

En cas d'erreur de build

Lire la première erreur réelle, pas seulement le résumé final.

Reproduire localement avec npm run build.

Corriger le fichier indiqué par le message.

Committer et pousser la correction sur main.

Relancer le déploiement Vercel.

7. Checklist annuelle

Les dates de l'accueil sont actualisées.

Les routes des nouvelles pages existent.

Les listes d'images sont actualisées.

L'annuaire des enseignants est actualisé.

Les salles et capacités sont vérifiées.

Les surveillances et convocations sont vérifiées.

Les exports PDF et CSV ont été testés.

npm run lint passe sans erreur.

npm run build passe sans erreur.

Le commit est envoyé sur main.

✓Mercel affiche un déploiement réussi.

✓Le site en ligne a été testé après déploiement.

✓évolution recommandée

À terme, déplacer les données dans une base de données ou un outil d'administration permettrait de modifier les sessions sans toucher au code. En attendant, cette procédure garantit une mise à jour contrôlée et réversible grâce à Git.